IT-arkitektur och IT Service Management

Lösnings- och mjukvaruarkitektur

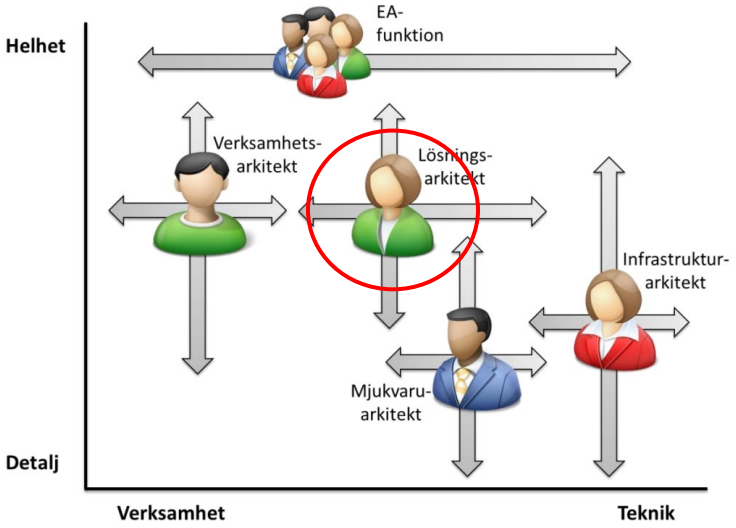
Planering

Vecka	Dag	Moment	Lärare
44	Tis	~~**Föreläsning** – Kursintroduktion & arkitektroller (kapitel 1, IASA-papper)~~	Sonny
	Fre	~~**Föreläsning** – Enterprisearkitektur (kapitel 2)~~	Ulf
	Sön	~~**Deadline Uppgift 1** – Arkitektroller~~	
45	Mån	~~**Föreläsning** – Verksamhetsarkitektur (kapitel 3)~~	Ulf
	-	~~**Föreläsning** – Modellering (inspelad)~~	Sonny
	Ons	~~**Karriärum & Handledning** – Uppgift 2~~	Sonny
	Sön	~~**Deadline Uppgift 2** – Modellering i ArchiMate~~	
46	Mån	**Föreläsning** – Lösnings- och mjukvaruarkitektur (kapitel 4-5)	Sonny
	Tis	**Föreläsning** – Infrastrukturarkitektur (forts. kapitel 4)	Sonny
	Sön	**Deadline Uppgift 3** – Rollen som infrastrukturarkitekt	
47	Ons	**Föreläsning** – ITIL del 1	Hannes
	Ons	**Seminarium 1 (ITSM-projektet)**	Hannes
48	Tor	**Föreläsning** – ITIL del 2	Hannes
	Tor	**Seminarium 2 (ITSM-projektet)**	Hannes
49	Ons	**Föreläsning** – ITIL del 3	Hannes
	Ons	**Seminarium 3 (ITSM-projektet)**	Hannes
50	Lör	**Tentamen**	
	-	Extra inlämning **Uppgift 1-3** **& Deadline** **ITSM-projektet**	

Kapitel 4

Lösningsarkitektur

Vad är lösningsarkitektur?



- Bryggan mellan verksamheten och tekniken
  - Har förmåga att omsätta verksamhetsarkitektens målbild i tekniska lösningar
  - Designar lösningar för att realisera verksamhetens behovsbild (**målarkitektur, TO-BE**)
- Ser till att det finns **integrationer** mellan olika system, system i samverkan
- Handlar ofta om mjukvara, men lösningsarkitektur har mer av ett helikopterperspektiv, vilket även inkluderar integration med infrastruktur
- Bör komma innan mjukvaruarkitektur

Lösningsarkitekten

- Finns en begreppsförvirring:
  - Kan vara en person som har ett ben i verksamheten, och ett i tekniken
  - Kan vara en person som har djup teknisk kunskap inom mjukvarulösningar
  - Kan vara en person som har djup teknisk kunskap inom mjukvaruintegration
  - Kan vara en person som har djup teknisk kunskap inom infrastruktur
  - AWS Solutions Architect (AWS-SA)
  - Cisco Certified Architect (CCAr)
  - Eller en kombination av ovanstående…

Lösningsarkitektur

- Handlar om att: *”…planera realiseringen av IT-lösningar baserat på verksamhetens behovsbild och med förutsättningar från existerande IT-tjänster i organisationen”*
- Handlar även om att få IT att samfungera vad gäller redan befintliga och framtida applikationer, system, tjänster och infrastruktur (**integrationer**)
- En lösning kan skapas på flera sätt:
  - **Använda existerande lösningar** (köpa ett helt färdigt system, t.ex. SaaS)
  - **Modifiera existerande lösningar** (ändra i köpt system, ändra konfiguration)
  - **<u>Utveckla nya lösningar</u>** (systemutveckling, mjukvaruarkitektens område)

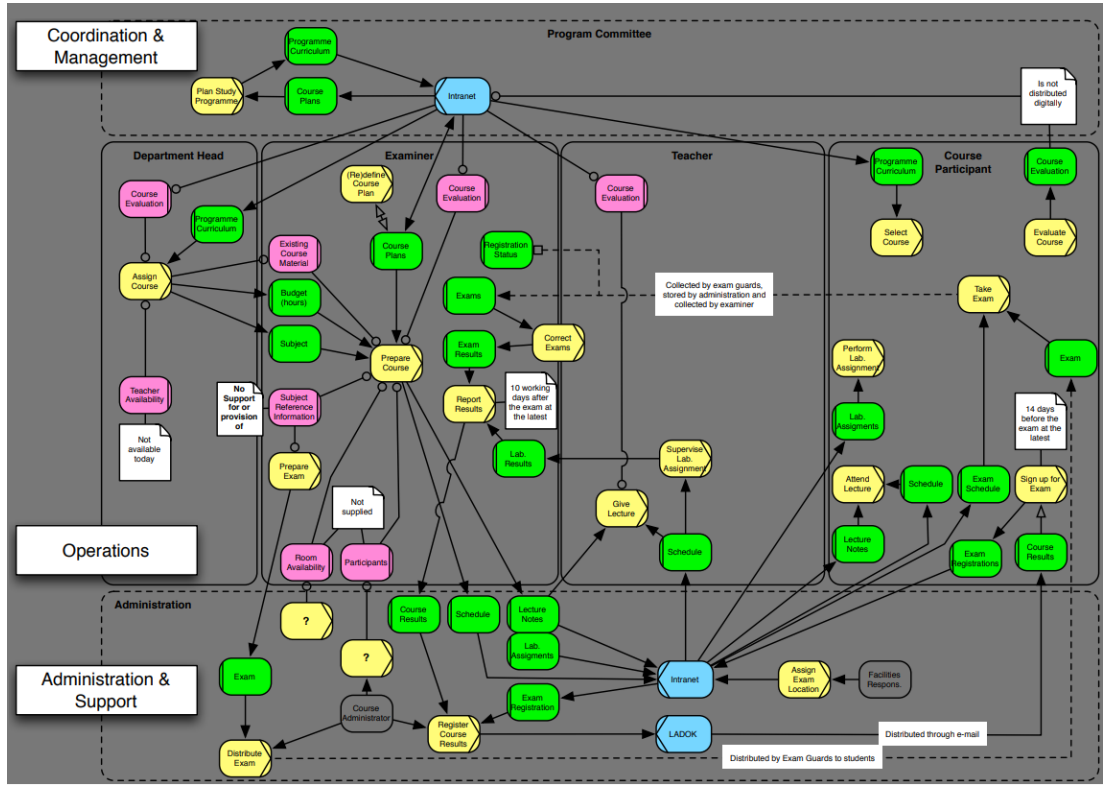
Val av lösning

- När vi utvecklar en ny lösning måste vi tidigt göra ett antal val gällande lösningsarkitekturen:
- **Plattform och språk**, t.ex. Java eller .NET?
- **Utvecklingsmetodik**, t.ex. Vattenfall eller Scrum, DevOps?
- **Metod för integration**, t.ex. EAI-nav eller REST?
- **Infrastruktur**, t.ex. on-premises eller cloud? Paas eller IaaS?

- När valen är gjorda ska systemet tas fram:

I föreläsningen gällande verksamhetsarkitektur…

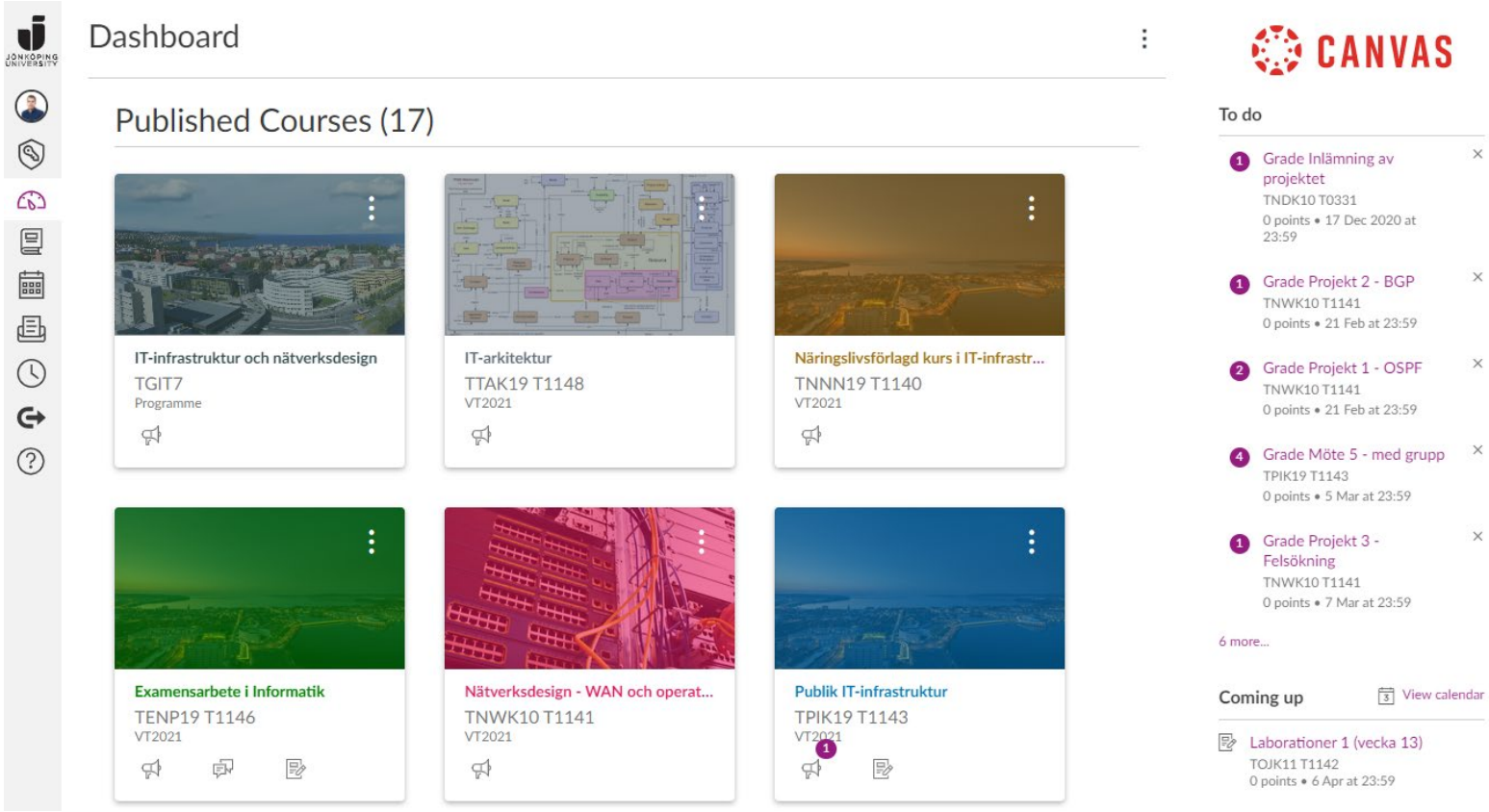
- …har Ulf visat en övergripande modell för hur en kurs genomförs på högskolan
- En lösningsarkitekt tar denna modell (kanske BPMN) och designar en teknisk lösning
- En vanlig modelleringsteknik för lösningsarkitekten kan då vara:



UML

- Många modeller som vi har beskrivit är ganska övergripande
- **Unified Modeling Language (UML)** är ett sätt att modellera system objektorienterat
- Bra utförd UML-modellering kan användas till:
  - Som stöd för mjukvaruutvecklare
  - För att automatgenerera mjukvara (kod)
  - För att testa mjukvara

En väl utförd UML-modellering som underlag till mjukvaruarkitekten genererar…



Integration

- Betänk att vi har utvecklat två fristående informationssystem:
  - **Canvas**, lärplattform för högskolor och universitet
  - **Ladok**, resultatrapportering för högskolor och universitet
- Att koppla ihop systemen med varandra brukar benämnas **integration**
  - **I bästa fall:** Integrationen är en del av grundarkitekturen
  - **I sämsta fall:** Integrationen görs som en efterkonstruktion
  - **I vanliga fall:** Man har tänkt på integration men inte lyckats helt
- Vad är en integration? Vad består den av?

Faktorer vid systemintegrationer

- När man pratar om integration mellan mjukvara, eller mellan system, finns det flera faktorer att ta hänsyn till, bl.a.:
  - **Grundl. datakommunikation:** Faktiska protokoll som används

Grundläggande datakommunikation

- Hur sker den faktiska överföringen av information mellan två system?
- Kan vara manuell filöverföring, t.ex. CSV-fil
- Schemalagda FTP-överföringar
- Simple Object Access Protocol (SOAP)
- REpresentational State Transfer (REST) som bygger på HTTP/HTTPS

Faktorer vid systemintegrationer

- När man pratar om integration mellan mjukvara, eller mellan system, finns det flera faktorer att ta hänsyn till, bl.a.:
  - **Grundl. datakommunikation:** Faktiska protokoll som används
  - **Syntax:** Informationsfälten som ska överföras, t.ex. fältet ”ECTS-Grade”
  - **Semantik:** Beskrivning av fälten ovan, begreppsdefinition
  - **Färskhetskrav:** Krav på att informationen är uppdaterad

Färskhetskrav

- Vilka krav har vi på att informationen är uppdaterad?
- Om anropet hämtar butikssaldo för dryck:
  - Hur mycket påverkar 5 nanosekunder?
  - Hur mycket påverkar 1 dag?
- Hur viktigt är det i en processindustri?
- I ett lönesystem?

Faktorer vid systemintegrationer

- När man pratar om integration mellan mjukvara, eller mellan system, finns det flera faktorer att ta hänsyn till, bl.a.:
  - **Grundl. datakommunikation:** Faktiska protokoll som används
  - **Syntax:** Informationsfälten som ska överföras, t.ex. fältet ”ECTS-Grade”
  - **Semantik:** Beskrivning av fälten ovan, begreppsdefinition
  - **Färskhetskrav:** Krav på att informationen är uppdaterad
  - **Juridisk kontext:** Vissa anrop kräver särskilda tillstånd, t.ex. journaler från sjukvården
  - **Säkerhet:** Skydd av känsliga uppgifter
  - **Övriga avtalsfrågor:** SLA, priser, etc.

Olika huvudtyper av integrationsflöden

- Man pratar om tre huvudsakliga typer:
- **Traditionell integration**, t.ex. manuell integration eller Enterprise Application Integration (EAI)
- **Tjänsteorienterad integration** – Service Oriented Architecture (SOA)
- **Händelsedriven integration** – Event Driven Architecture (EDA)

Traditionella integrationsflöden

- Fungerar bra när färskhetskravet är lågt
- Bygger ofta på enkel filöverföring mellan två system, t.ex. en CSV-fil som laddas ned från Canvas och laddas upp i Ladok (**manuell integration**)
- Kan även bygga på överföringar via FTP (SFTP/FTPS) med manuell aktivering
- **Enterprise Application Integration (EAI-nav)** kan även användas för detta
  - Automatiserar manuella och traditionella integrationsflöden
  - Ett exempel är: Microsoft BizTalk
- **EAI** är en typ av **middleware** (mellanprogramvara)
- Blir lätt ”spaghetti-integration”

Service Oriented Architecture (SOA)

- **Service Oriented Architecture (SOA)** bygger på tjänsteorientering
- Stark trend under början av 2000-talet som fortfarande lever kvar
- Huvudsyftet är att utveckla flexibla och återanvändningsbara moduler (**microservices**, nu ofta via **containers**)
  - Modularisera och dela upp mjukvara
  - Tillföra övergripande processlogik (vad görs i verksamheten?)
  - Ta fram mönster för hur moduler ska kommunicera (integration)
- Huvudsakligen används protokollen SOAP och REST för integrationen

SOAP

- Var tidigare en akronym för **Simple Object Access Protocol (SOAP)**
  - **Extensibility** – Enkelt att skapa tillägg och varianter
  - **Neutrality** – Fungerar över flera protokoll, t.ex. HTTP, SMTP, etc.
  - **Independence** – Fungerar mellan olika systemstandarder, och programspråk
- SOAP använder huvudsakligen HTTP för att skicka meddelanden mellan system/applikationer, ofta via XML
  - Innebär att ”adressen” för integrationen är en URL
- Ofta används begreppet **Web Services** när HTTP och URLs används för kommunikation som är maskin-till-maskin, t.ex. SOAP, men även REST

SOAP-anrop

```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPrice>
    <m:StockName>IBM</m:StockName>
  </m:GetStockPrice>
</soap:Body>

</soap:Envelope>
```

SOAP-svar

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn


<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPriceResponse>
    <m:Price>34.5</m:Price>
  </m:GetStockPriceResponse>
</soap:Body>

</soap:Envelope>
```

REST

- **REpresentational State Transfer** (**REST**) skiljer sig från SOAP genom :
  - **Ej ett eget protokoll**, bara ett ramverk (en arkitektur) för att skicka information
  - **Bygger helt på HTTP** och säkerhet är därför via SSL/TLS
  - **Helt stateless**, bygger inga sessioner och är inte medveten om klienten
- En **Web Service** som körs enligt REST brukar kallas för RESTful, och ger precis som SOAP integration mellan olika system
  - ***Web service*** vs. ***microservice*** vs. ***container***?
- Payload är oftast ren JSON-data
  - Jämför med SOAP som har en egen header

REST-anrop

```
GET /people HTTP/1.1
Host: www.namefinder.com
Content-Type: text/html; charset=utf-8
Content-Length: nnn
```

Hämtar tekniskt sett endast informationen på URLen
http://www.namefinder.com/people

REST-svar

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: nnn

{
  "first name": "Alice",
  "last name": "Smith",
  "age": 25,
  "address": {
    "street address": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal code": "10021"
  }
}
```

REST-anrop

```
GET /?format=json HTTP/1.1
Host: api.ipify.org
```

Hämtar tekniskt sett endast informationen på URLen
http://api.ipify.org/?format=json

REST-svar

```
HTTP/1.1 200 OK
Content-Length: 21
Content-Type: application/json

{
  “ip": “193.10.31.17",
}
```

Event Driven Architecture (EDA)

- Händelsedriven integration bygger på att direkt när ett värde har förändrats sprids informationen till berörda system
- Kompletterar SOA i de fall som en händelse ska aktivera ett informationsflöde:
  - Adressändring hos Skatteverket
  - Ej besiktigad bil
  - Slutsåld vara på Systembolaget
  - Rättad Inspera-tenta

Förvaltningsbarhet och integrationer

- Själva utvecklingen av systemen och framtagandet av dess integrationer är av engångskaraktär, längst befinner sig dock mjukvaran i **förvaltning/drift** (största delen av kostnaden):
  - Vad behöver göras dagligen, veckovis, månadsvis, årsvis för att systemet ska fungera?
  - Går det att modifiera systemet om behoven förändras?
  - Support till användare som använder systemet/applikationen
- En stor del av förvaltningsfasen är att hantera den dagliga driften och användarsupport (**IT Operations**, IT Ops, ITSM, IT-infrastruktur)

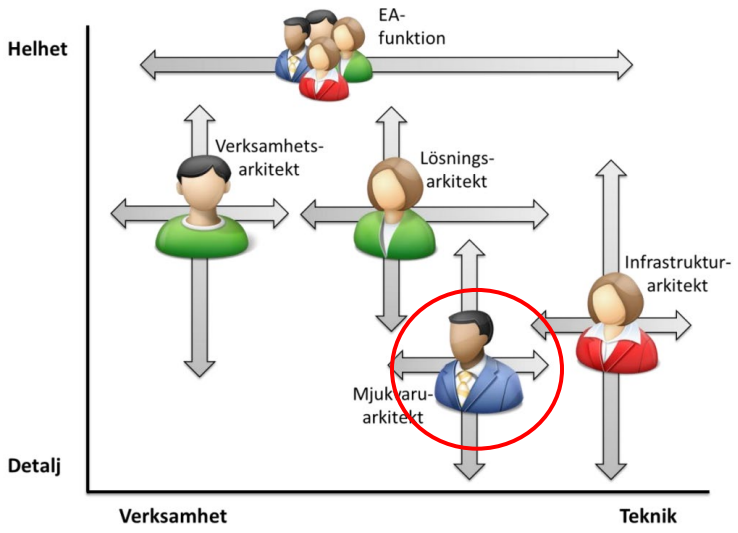
Hur påverkar detta IT-infrastrukturen?

Det tar vi reda efter en snabb detour förbi området **mjukvaruarkitektur**

Kapitel 5

Mjukvaruarkitektur

Vad är mjukvaruarkitektur?

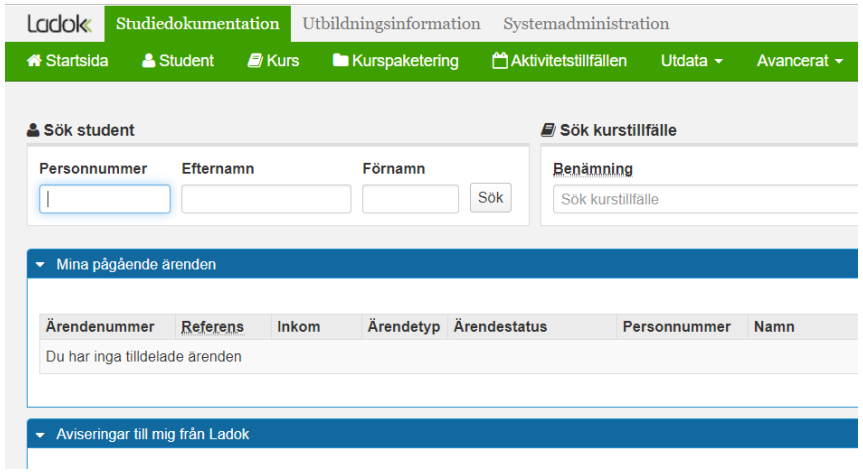


- Hanterar mjukvaran/system och utvecklare
- Ansvarig för design, struktur av mjukvaru-utvecklingen, funktionella krav
- Ofta del i ett projekt(ledare), eller med ansvar för ett specifikt IT-system eller produkt
- Jämfört med lösningsarkitektur som hanterar integration mellan *flera* system, har mjukvaruarkitektur ofta fokus på *ett* system
- Kräver god kompetens inom mjukvaruutveckling

Vad är mjukvara?

print(”Hello, World!")

- Kod som utför vissa uppgifter i en dator
- Programvara
- Datorprogram
- Webbsida
- Informationssystem
- IT-system
- Applikation
- App



Vad är mjukvaruarkitektur?

- Fokus på arkitekturen hos (ett) system: hur det fungerar i både **detalj** och **övergripande**
- Arbete med **kvalitetsattribut** hos mjukvara
- **Funktionella krav** och **icke-funktionella krav**
- **Design och struktur** av mjukvaruutveckling
- **Projektledning** av mjukvaruprojekt

Kvalitetsattribut

- **Användarbarhet och estetik**
  - Användartester, fokus på ”front-end”
- **Anpassningsbarhet**
  - Hantera förändringar, konfiguration
- **Driftsäkerhet**
  - Hur hanterar systemet stor belastning?
- **Integrationsmöjligheter**
  - Hur väl går det att integrera med andra system?

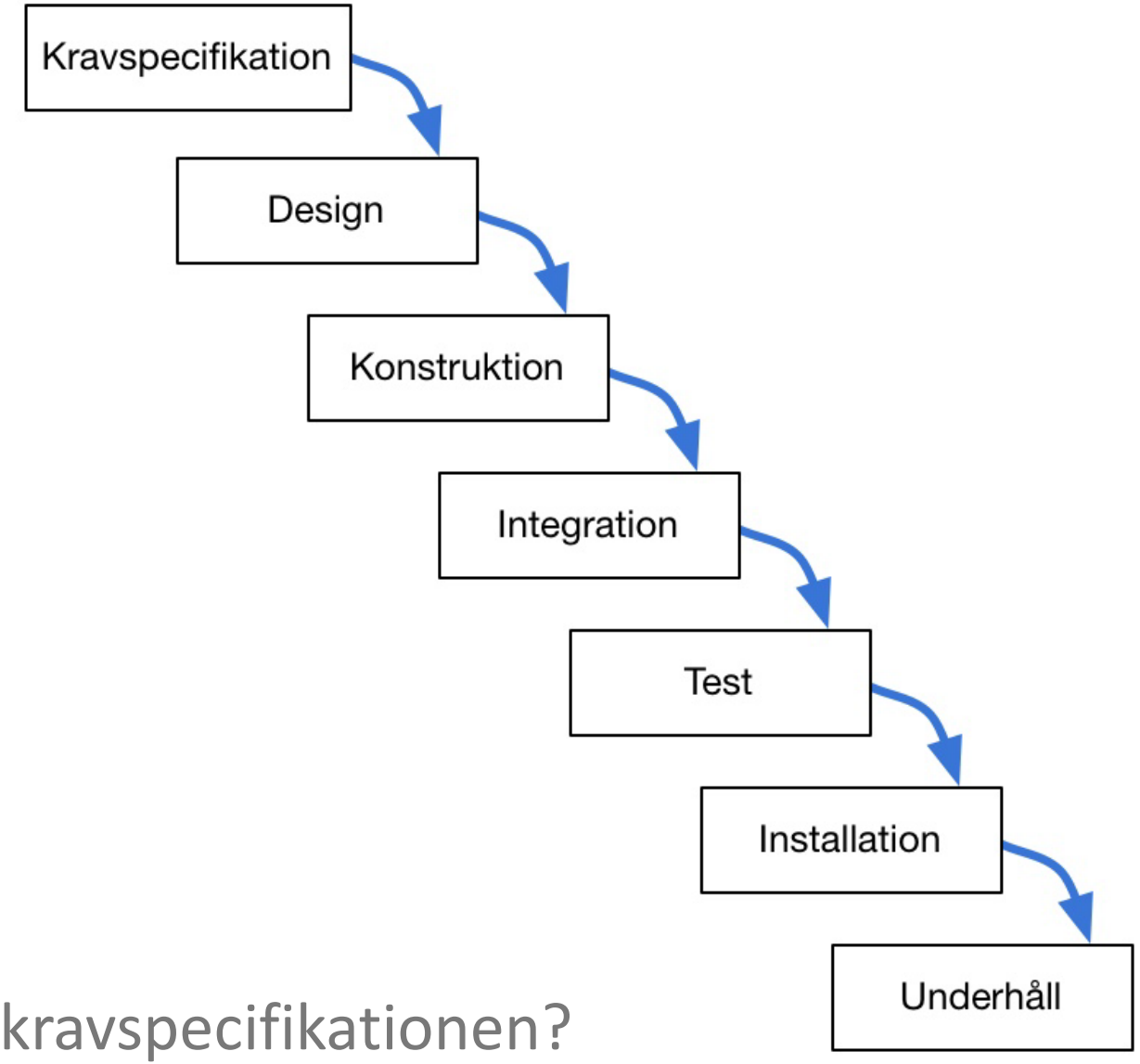
Funktionella vs. icke-funktionella krav

- Första steget i processen är oftast att samla in krav från olika aktörer
- Kraven brukar delas in i två grupper:
- **Funktionella krav** – Kopplat till specifika funktioner, t.ex. ett fakturasystem ska ha funktioner för betalning
- **Icke-funktionella krav** – Inte kopplat till specifika funktioner som ska gå att utföra, istället kopplat mot aspekter som säkerhet, prestanda, tillgänglighet (kvalitet)

Hur utvecklas mjukvara?

- Ett optimalt förfarande innefattar verksamhets-, lösnings-, mjukvaru- och infrastrukturarkitekter
- Krav -> Utveckling -> Test -> Implementation
- Finns flera systemutvecklingsmetoder:
  - **Vattenfall**
  - **RUP**
  - **Agilt**
  - Lean
  - TDD

Vattenfallsmodellen



- Vad händer om du gör fel i kravspecifikationen?

https://sv.wikipedia.org/wiki/Vattenfallsmodellen

Miljardfiasko – nu skrotas Skolplattformen

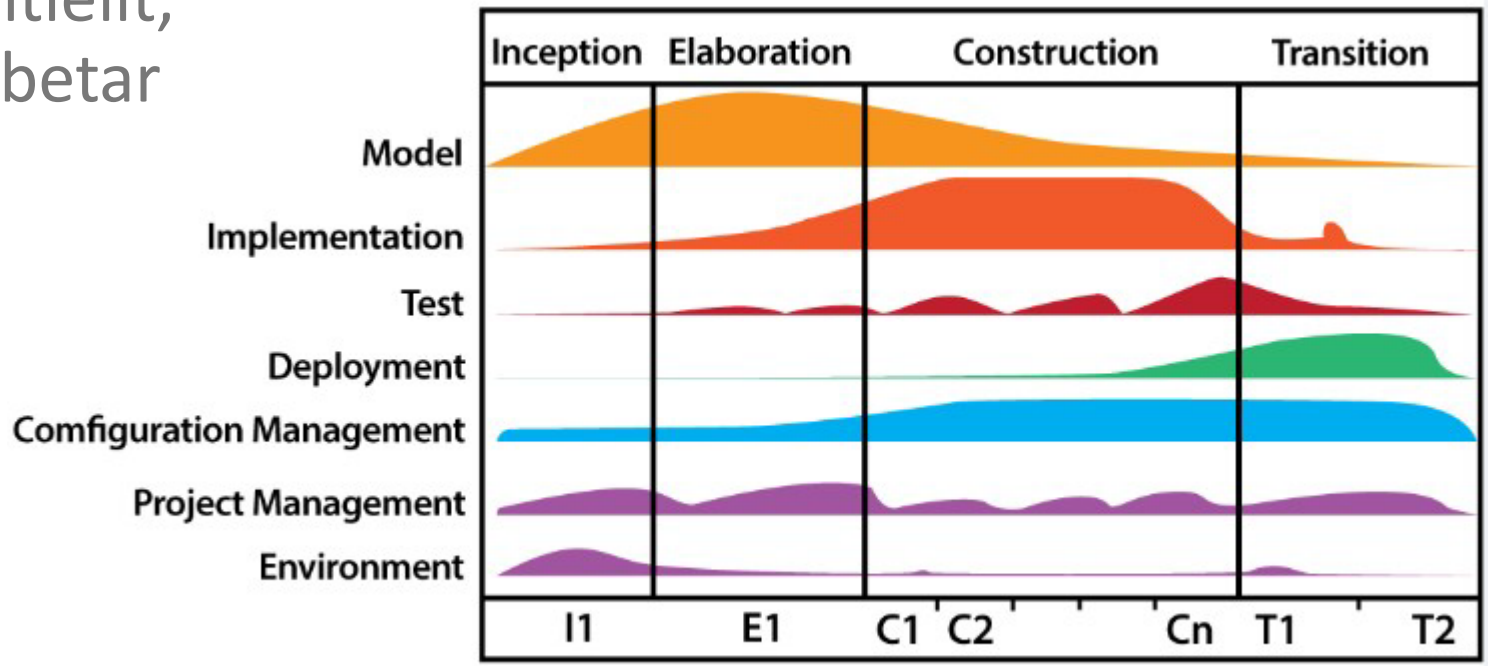
DIGITALA VERKTYG Stockholms stad har flera gånger varit i blåsväder för Skolplattformen. Dels har plattformen kostat över en miljard kronor – dels är den komplicerad att använda och har läckt information. Nu ersätts den med enklare verktyg.

EMMA OLSSON
19 APR 2023

https://www.vilarare.se/nyheter/digitala-verktyg/dyr-och-dalig--nu-slipper-lararna-skolplattformen/

Rational Unified Process (RUP)

- Istället för att arbeta sekventiellt, likt vattenfalls-modellen, arbetar RUP i olika faser samtidigt

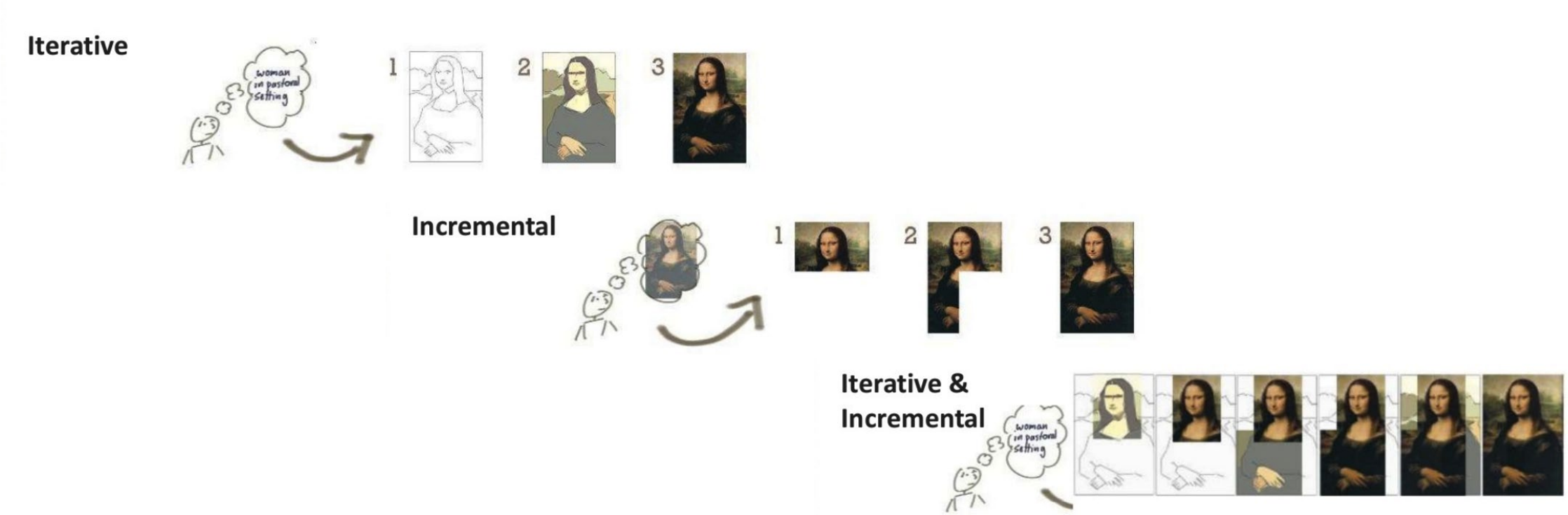


Agila metoder

- Har blivit det vanligaste sättet att arbeta med mjukvaruprojekt
- Agila manifestet (utdrag):
  - Högsta prioritet att tillfredsställa kunden genom **tidig och kontinuerlig leverans**
  - Välkomnande inför **förändrade krav** (även sent)
  - **Verksamhets**-kunniga ska arbeta tätt med **teknik**-kunniga
  - Satsa på personer som arbetar i projektet, **arbeta tillsammans, på plats**
  - Enkelhet är viktigt, gör alltid **enklaste möjliga lösningen** som löser uppgiften
- Vanliga agila ramverk är **Scrum**, Kanban, XP

Scrum

- Bygger på arbete som är **iterativt** och **inkrementellt**

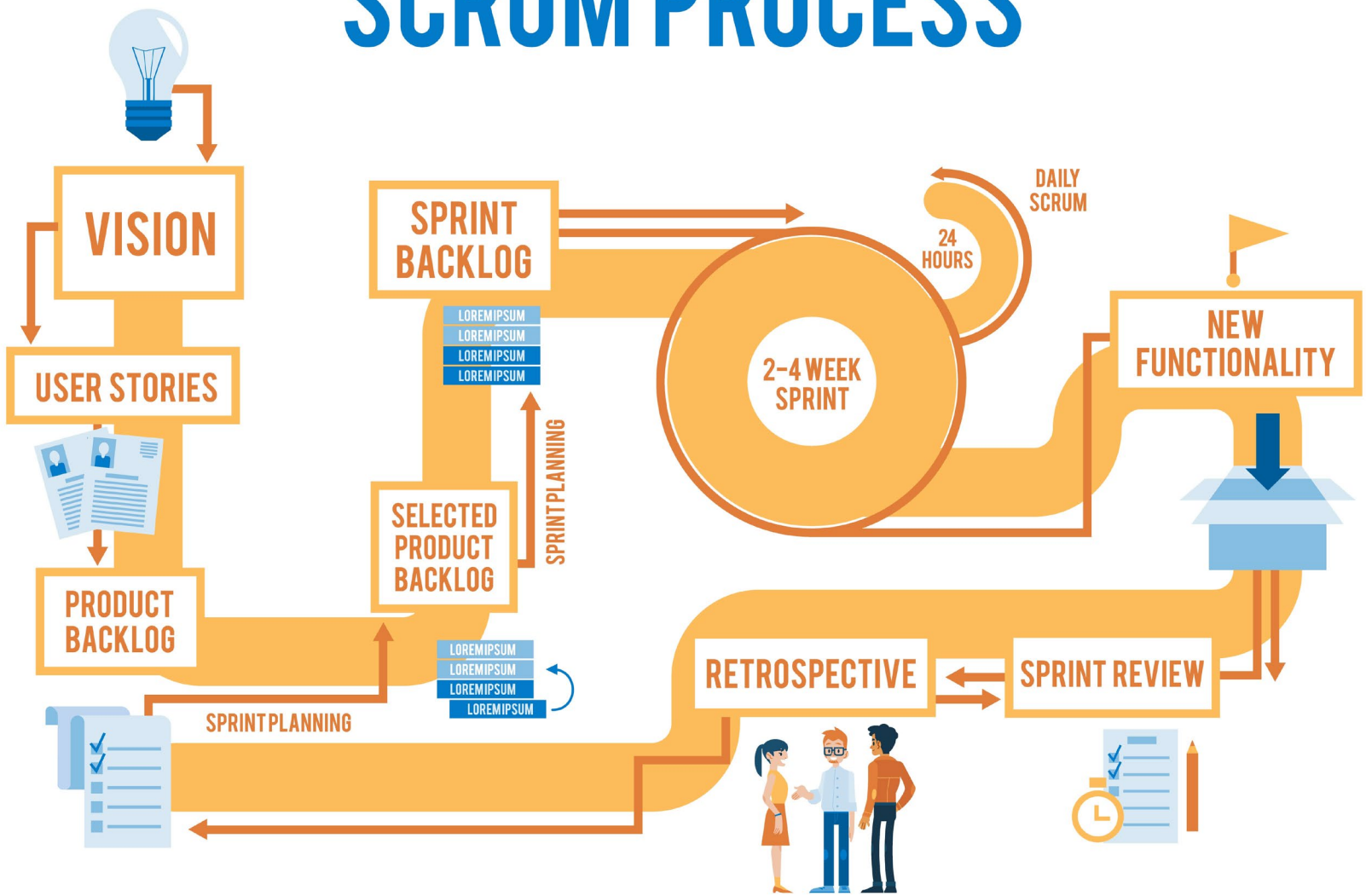


https://medium.com/@SupriyaL/why-and-how-to-use-agile-methodology-for-an-online-food-delivery-service-503ad5cf1941

Roller i Scrum

- **Produktägare:** En person som är ansvarig för:
  - **Product Backlog** (”fullständig” funktionalitet som önskas av mjukvaran)
- **Utvecklingsteam:** Grupp med utvecklare, ofta multi-disciplinärt (frontend, backend, db, etc.)
  - Arbetar med en **Sprint Backlog** som är den funktionalitet som ska bli färdig under en sprint (ca. 2-4 veckor)
- **Scrum Master:** Säkerställer att gruppen följer Scrum-principer och uppnår sina mål, dock ej projektledare… (enligt det agila manifestet)

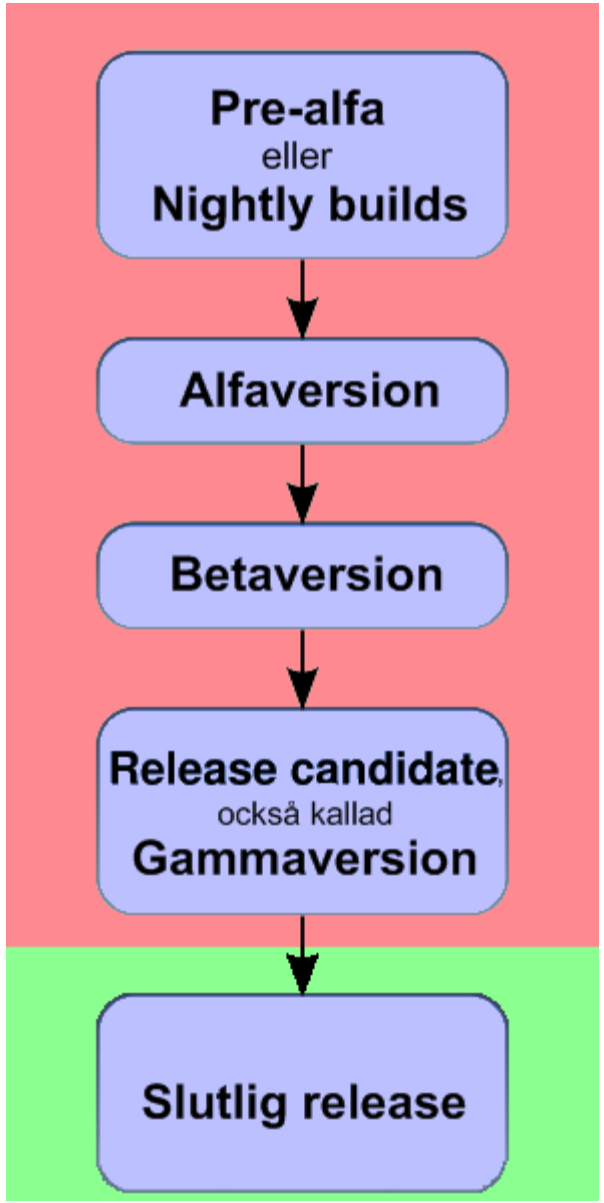
SCRUM PROCESS



Läs mer: https://www.ntaskmanager.com/blog/newbies-guide-to-scrum-project-management-101/

Leverans av mjukvara (*Release Management*)

- Ett av de sista stegen i alla modeller
- Kan vara tätt sammankopplat mot ITIL-processer
  - *Change Management*
  - *Release and Deployment Management*
- I och med agila arbetsätt har detta arbetssättet ändrats (i vissa organisationer)



https://sv.wikipedia.org/wiki/Release_management

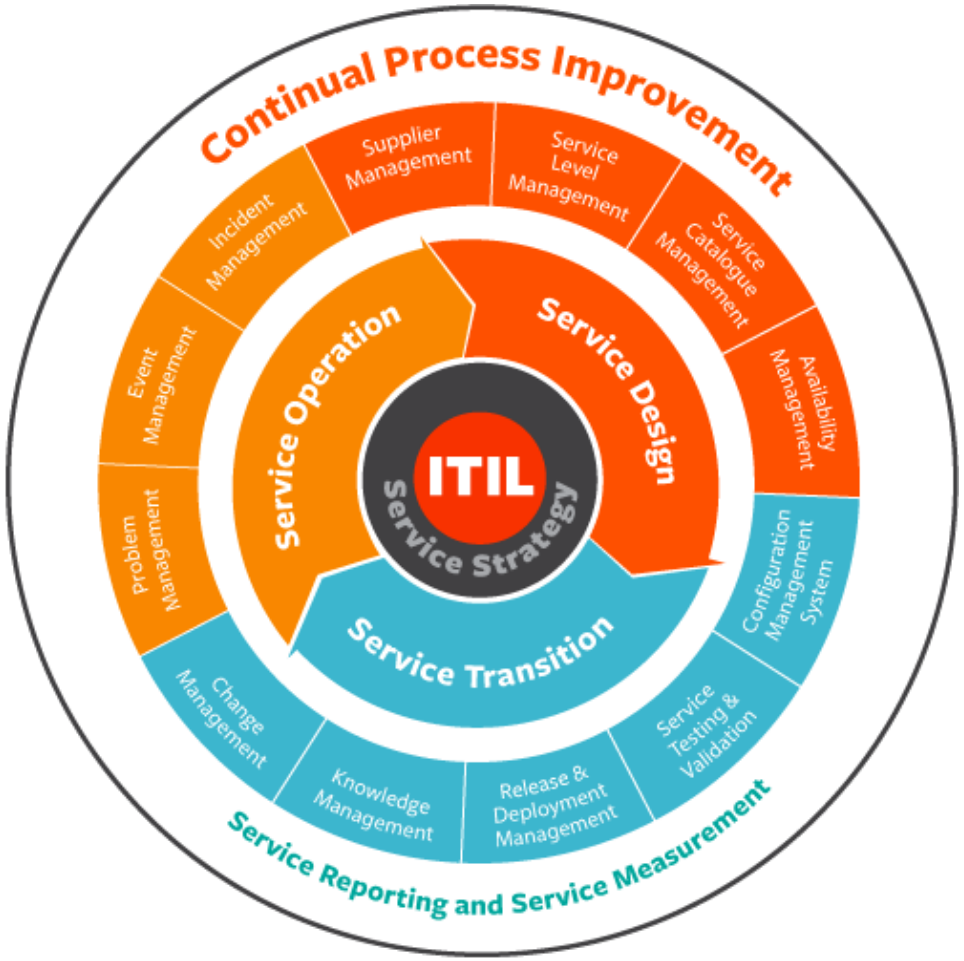
Förvaltning och administration av mjukvara

- När mjukvaran levereras och driftsatts når mjukvaran förvaltningsfasen
- **Systemförvaltning** är ansvaret för dagliga drift och administration, och även ett långsiktigt perspektiv på användningen
  - Leda vidareutveckling av mjukvaran
  - Löpande kontakt med användare, administratörer, utvecklare
  - Göra användarmanualer, hålla utbildningar
  - Hantera behörigheter och avtal
  - Övergripande ansvar för säkerhet
- Två modeller för systemförvaltning är **PM3** och **ITIL**

Förvaltning och administration av mjukvara

- **Systemadministration** kan vara samma roll som systemförvaltning, men skiljer sig ofta genom:
  - Större teknisk fokus (där förvaltning ofta har ett större verksamhetsfokus)
  - Ansvarig för driftsättningen
  - Regelbundet underhåll, t.ex. uppdateringar, patchningar, backuper
  - Övervakning av systemet
  - Felsökning och åtgärder (t.ex. via servicedesk)
  - Arbetar ofta med flera system, och övrig IT-infrastruktur
  - Klienthantering

Förvaltning och administration



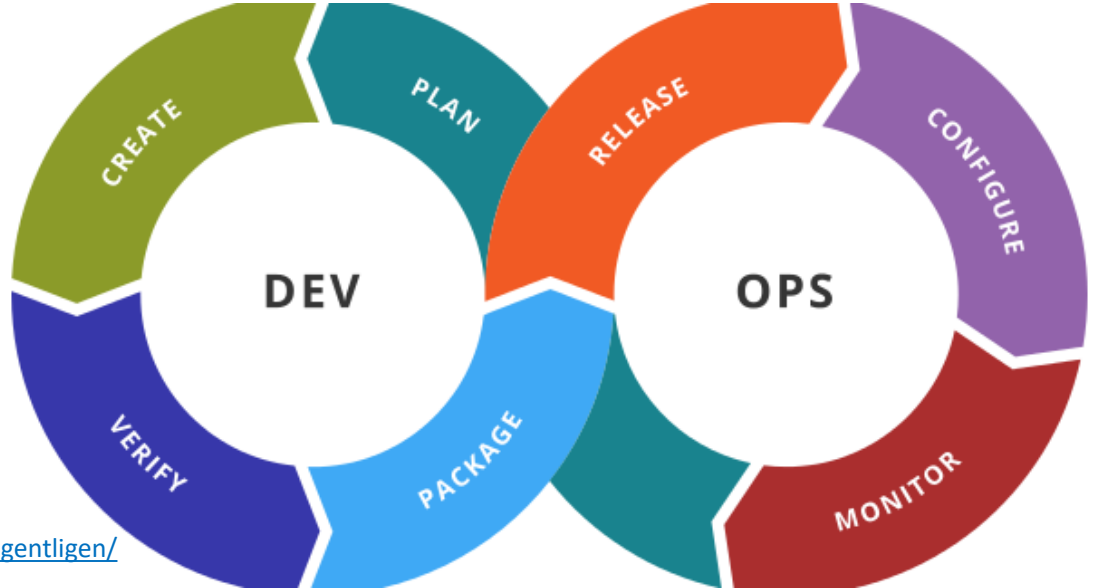
https://networkdirection.net/articles/network-theory/networklifecycle/

Nya sätt att leverera mjukvara

- Tidigare utvecklades system, och efter tester togs systemet i drift. Vid uppdateringar/migreringar:
  - Meddela om servicefönster
  - Stäng av gamla systemet
  - Migrera data till nya systemet (databaskopplingar)
  - Koppla på nya systemet
  - Testa, och eventuell ”roll-back”
- Ett nytt sätt att tänka kring både utveckling och leverans av mjukvara är **DevOps** och **CI/CD** som har blivit vanligare i och med agil mjukvaruutveckling

DevOps

- Traditionellt har mjukvara utvecklats (Development, **Dev**) och sedan levererats, förvaltats och underhållits (IT Operations, **Ops**)
- De agila arbetsmetoderna har gjort att Dev och Ops måste arbeta tätare, lösningen: **DevOps**
- Finns i olika utföranden:
  - Samma avdelning, olika kompetenser inom Dev respektive Ops
  - Samma avdelning, heltäckande kompetenser inom DevOps
  - Olika avdelningar, tätt samarbete



https://www.frontit.se/inspiration-kunskap/artiklar/devops-ci-cd-vad-betyder-det-egentligen/

CI/CD - Continuous Integration/Continuous Delivery

- Nära ihopkopplat med DevOps-konceptet
- **Continuous Integration**
  - Ihopslagning av kod mellan utvecklare flera gånger per dag (istället för i slutet)
  - Ofta via ett versionshanteringssystem, t.ex. Git
- **Continuous Delivery/Deployment**
  - Istället för stora servicefönster varje månad, levereras ny funktionalitet till användare kontinuerligt i form av *microservices*, i vissa fall automatiserat
- En uppsatt miljö kring integration av kod, automatiserade tester och driftsättning brukar kallas för **CI/CD pipeline**

CI/CD med Docker och Kubernetes

- En mer avancerad **CI/CD-pipeline** involverar även att den nya funktionen som utvecklas också driftsätts automatiserat (deployment)
  1. Utveckling av ny funktion
  2. Automatiskt: Testning
  3. Automatiskt: Skapa en Docker-container
  4. Automatiskt: Skicka in container i Kubernetes
- **Kubernetes** är ett verktyg (öppen källkod) för orkestrering av containrar (jämför vCenter för VMs)

https://semaphoreci.com/blog/cicd-pipeline

Rollen för infrastruktur (*IT Ops*) i DevOps-miljöer

…undersöker vi nästa vecka!

kahoot.it

JÖNKÖPING UNIVERSITY

School of Engineering